

# Отчет по лабораторной работе №26 по курсу «Алгоритмы и структуры данных»

Студент группы М8О-116БВ-25 Сиухин Ярослав Алексеевич, № по списку 22

Контакты www, e-mail: zaq2007q@gmail.com

Работа выполнена: «03» июня 2026 г.

Преподаватель: Речинская Ангелина Юрьевна

Отчет сдан «\_\_» \_\_\_\_\_ 2026 г., итоговая оценка \_\_\_\_\_

Подпись преподавателя \_\_\_\_\_

|         |                                                                                                                                                                               |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Тема    | Реализация стека, очереди и кольцевого буфера.                                                                                                                                |
| Цель    | Разработать набор динамических структур данных и протестировать их основные операции.                                                                                         |
| Задание | Реализовать стек с динамическим массивом и итератором, сортировку стека слиянием, очередь на двух стеках и кольцевой буфер с операциями добавления и удаления с обеих сторон. |
| Файлы   | /Users/siuxin_yaroslav/Desktop/projectsLabsC/second_semestr/second_semestr_algorithm_and_stryktyr_dann/lr26                                                                   |

## Оборудование и программное обеспечение

- Компьютер: Apple Silicon, архитектура arm64.
- Операционная система: macOS 15.7.3.
- Интерпретатор команд: bash 3.2.57.
- Система программирования: Apple clang 17.0.0, стандарт C11.
- Редактор текстов: nano / среда разработки и терминал macOS.

## Идея, метод и алгоритм

Стек хранится в расширяемом массиве, очередь реализуется через два стека, а кольцевой буфер использует индексы head/tail и перераспределение памяти при заполнении.

1. Инициализировать стек с начальной емкостью и увеличивать ее через realloc при заполнении.
2. Реализовать итератор стека для последовательного обхода от дна к вершине.
3. Для сортировки стека разделить данные на две части, рекурсивно отсортировать их и слить в порядке возрастания.
4. Для очереди добавлять элементы во входной стек, а при извлечении перекладывать их в выходной стек.
5. Для кольцевого буфера поддерживать индексы начала, конца, размер и емкость.

6. Проверить операции push/pop/reek и освобождение памяти для каждой структуры.

# Сценарий выполненной работы

## Файл: main.c

```
#include <stdio.h>
#include "stack.h"
#include "stack_sort.h"
#include "queue.h"
#include "ringbuf.h"

void print_stack(const Stack *s) {
    StackIterator it = stack_iterator_begin(s);
    int value;
    printf("Стек (дно -> вершина): ");
    while (stack_iterator_next(&it, &value)) printf("%d ", value);
    printf("\n");
}

int main() {
    printf("=== Тестирование стека ===\n");
    Stack s;
    stack_init(&s);
    int test_data[] = {42, 17, 8, 99, 23, 5, 67, 31};
    int n = sizeof(test_data) / sizeof(test_data[0]);
    printf("Вталкиваю в стек: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", test_data[i]);
        stack_push(&s, test_data[i]);
    }
    printf("\n");
    print_stack(&s);
    stack_merge_sort(&s);
    printf("После сортировки слиянием:\n");
    print_stack(&s);
    printf("Извлечение: ");
    int val;
    while (stack_pop(&s, &val)) printf("%d ", val);
    printf("\n\n");
    stack_destroy(&s);

    printf("=== Тестирование очереди ===\n");
    Queue *q = queue_create();
    int queue_data[] = {10, 20, 30, 40, 50};
    n = sizeof(queue_data) / sizeof(queue_data[0]);
    printf("Добавляю в очередь: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", queue_data[i]);
        queue_enqueue(q, queue_data[i]);
    }
    printf("\nРазмер очереди: %d\n", queue_size(q));
    printf("Извлечение из очереди: ");
    while (queue_dequeue(q, &val)) {
        printf("%d ", val);
    }
    printf("\nОчередь пуста: %s\n\n", queue_is_empty(q) ? "да" : "нет");
    queue_destroy(q);

    printf("=== Тестирование кольцевого буфера ===\n");
    RingBuf rb;
    ringbuf_init(&rb);

    printf("Заполняю буфер (push_back): 1 2 3 4 5\n");
    ringbuf_push_back(&rb, 1);
    ringbuf_push_back(&rb, 2);
    ringbuf_push_back(&rb, 3);
    ringbuf_push_back(&rb, 4);
    ringbuf_push_back(&rb, 5);
    printf("Размер: %d, peek_front: ", ringbuf_size(&rb));
    int f, b;
    ringbuf_peek_front(&rb, &f);
    printf("%d, peek_back: ", f);
    ringbuf_peek_back(&rb, &b);
    printf("%d\n", b);

    printf("Добавляю в начало (push_front): 0\n");
    ringbuf_push_front(&rb, 0);
}
```

```

printf("Размер: %d, peek_front: ", ringbuf_size(&rb));
ringbuf_peek_front(&rb, &f);
printf("%d\n", f);

printf("Извлекаю с конца (pop_back): ");
ringbuf_pop_back(&rb, &val);
printf("%d\n", val);
printf("Размер: %d, peek_back: ", ringbuf_size(&rb));
ringbuf_peek_back(&rb, &b);
printf("%d\n", b);

printf("Извлекаю спереди (pop_front): ");
ringbuf_pop_front(&rb, &val);
printf("%d\n", val);
printf("Размер: %d, peek_front: ", ringbuf_size(&rb));
ringbuf_peek_front(&rb, &f);
printf("%d\n", f);

printf("Очищаю буфер: ");
while (!ringbuf_is_empty(&rb)) {
    ringbuf_pop_front(&rb, &val);
    printf("%d ", val);
}
printf("\nБуфер пуст: %s\n", ringbuf_is_empty(&rb) ? "да" : "нет");

ringbuf_destroy(&rb);
return 0;
}

```

## Файл: stack.c

```
#include "stack.h"
#include <stdlib.h>

#define INITIAL_CAPACITY 4

void stack_init(Stack *s) {
    s->data = (int*) malloc(INITIAL_CAPACITY * sizeof(int));
    s->top = -1;
    s->capacity = INITIAL_CAPACITY;
}

void stack_destroy(Stack *s) {
    free(s->data);
    s->data = NULL;
    s->top = -1;
    s->capacity = 0;
}

bool stack_is_empty(const Stack *s) {
    return s->top == -1;
}

bool stack_is_full(const Stack *s) {
    return s->top + 1 >= s->capacity;
}

static bool stack_resize(Stack *s) {
    int new_capacity = s->capacity * 2;
    int *new_data = (int*) realloc(s->data, new_capacity * sizeof(int));
    if (!new_data) return false;
    s->data = new_data;
    s->capacity = new_capacity;
    return true;
}

bool stack_push(Stack *s, int value) {
    if (stack_is_full(s) && !stack_resize(s)) {
        return false;
    }
    s->data[++(s->top)] = value;
    return true;
}

bool stack_pop(Stack *s, int *out_value) {
    if (stack_is_empty(s)) return false;
    *out_value = s->data[(s->top)--];
    return true;
}

bool stack_peek(const Stack *s, int *out_value) {
    if (stack_is_empty(s)) return false;
    *out_value = s->data[s->top];
    return true;
}

int stack_size(const Stack *s) {
    return s->top + 1;
}

StackIterator stack_iterator_begin(const Stack *s) {
    StackIterator it;
    it.stack = s;
    it.current = 0;
    return it;
}

bool stack_iterator_has_next(const StackIterator *it) {
    return it->current <= it->stack->top;
}

bool stack_iterator_next(StackIterator *it, int *out_value) {
    if (!stack_iterator_has_next(it)) return false;
    *out_value = it->stack->data[it->current];
    it->current++;
    return true;
}
```

}

## Файл: stack\_sort.c

```
#include "stack_sort.h"
#include <stdlib.h>

void stack_merge_sort(Stack *s);

static void split_stack(Stack *src, Stack *left, Stack *right) {
    int size = stack_size(src);
    int half = size / 2;

    StackIterator it = stack_iterator_begin(src);
    int value;
    for (int i = 0; i < half; i++) {
        stack_iterator_next(&it, &value);
        stack_push(left, value);
    }
    while (stack_iterator_next(&it, &value)) {
        stack_push(right, value);
    }

    while (stack_pop(src, &value)) {}
}

static void merge_sorted_stacks(Stack *a, Stack *b, Stack *result) {
    StackIterator it_a = stack_iterator_begin(a);
    StackIterator it_b = stack_iterator_begin(b);
    int val_a, val_b;
    bool has_a = stack_iterator_next(&it_a, &val_a);
    bool has_b = stack_iterator_next(&it_b, &val_b);

    while (has_a && has_b) {
        if (val_a <= val_b) {
            stack_push(result, val_a);
            has_a = stack_iterator_next(&it_a, &val_a);
        } else {
            stack_push(result, val_b);
            has_b = stack_iterator_next(&it_b, &val_b);
        }
    }
    while (has_a) {
        stack_push(result, val_a);
        has_a = stack_iterator_next(&it_a, &val_a);
    }
    while (has_b) {
        stack_push(result, val_b);
        has_b = stack_iterator_next(&it_b, &val_b);
    }
}

void stack_merge_sort(Stack *s) {
    if (stack_size(s) <= 1) return;

    Stack left, right;
    stack_init(&left);
    stack_init(&right);

    split_stack(s, &left, &right);

    stack_merge_sort(&left);
    stack_merge_sort(&right);

    Stack merged;
    stack_init(&merged);
    merge_sorted_stacks(&left, &right, &merged);

    Stack temp;
    stack_init(&temp);
    int value;
    while (stack_pop(&merged, &value)) {
        stack_push(&temp, value);
    }
    while (stack_pop(&temp, &value)) {
        stack_push(s, value);
    }

    stack_destroy(&temp);
}
```

```

    stack_destroy(&merged);
    stack_destroy(&left);
    stack_destroy(&right);
}

```

## Файл: queue.c

```

#include "queue.h"
#include "stack.h"
#include <stdlib.h>

struct Queue {
    Stack in;
    Stack out;
};

Queue* queue_create(void) {
    Queue *q = (Queue*) malloc(sizeof(Queue));
    if (q) {
        stack_init(&q->in);
        stack_init(&q->out);
    }
    return q;
}

void queue_destroy(Queue *q) {
    if (q) {
        stack_destroy(&q->in);
        stack_destroy(&q->out);
        free(q);
    }
}

bool queue_enqueue(Queue *q, int value) {
    return stack_push(&q->in, value);
}

bool queue_dequeue(Queue *q, int *out_value) {
    if (queue_is_empty(q)) return false;

    if (stack_is_empty(&q->out)) {
        int val;
        while (stack_pop(&q->in, &val)) {
            stack_push(&q->out, val);
        }
    }
    return stack_pop(&q->out, out_value);
}

bool queue_is_empty(const Queue *q) {
    return stack_is_empty(&q->in) && stack_is_empty(&q->out);
}

int queue_size(const Queue *q) {
    return stack_size(&q->in) + stack_size(&q->out);
}

```



## Файл: ringbuf.c

```
#include "ringbuf.h"
#include <stdlib.h>

#define INITIAL_CAPACITY 4

void ringbuf_init(RingBuf *rb) {
    rb->data = (int*) malloc(INITIAL_CAPACITY * sizeof(int));
    rb->head = 0;
    rb->tail = 0;
    rb->size = 0;
    rb->capacity = INITIAL_CAPACITY;
}

void ringbuf_destroy(RingBuf *rb) {
    free(rb->data);
    rb->data = NULL;
    rb->head = 0;
    rb->tail = 0;
    rb->size = 0;
    rb->capacity = 0;
}

bool ringbuf_is_empty(const RingBuf *rb) {
    return rb->size == 0;
}

bool ringbuf_is_full(const RingBuf *rb) {
    return rb->size == rb->capacity;
}

int ringbuf_size(const RingBuf *rb) {
    return rb->size;
}

static bool ringbuf_resize(RingBuf *rb) {
    int new_capacity = rb->capacity * 2;
    int *new_data = (int*) malloc(new_capacity * sizeof(int));
    if (!new_data) return false;

    for (int i = 0; i < rb->size; i++) {
        new_data[i] = rb->data[(rb->head + i) % rb->capacity];
    }
    free(rb->data);
    rb->data = new_data;
    rb->head = 0;
    rb->tail = rb->size;
    rb->capacity = new_capacity;
    return true;
}

bool ringbuf_push_back(RingBuf *rb, int value) {
    if (ringbuf_is_full(rb) && !ringbuf_resize(rb)) {
        return false;
    }
    rb->data[rb->tail] = value;
    rb->tail = (rb->tail + 1) % rb->capacity;
    rb->size++;
    return true;
}

bool ringbuf_pop_front(RingBuf *rb, int *out) {
    if (ringbuf_is_empty(rb)) return false;
    *out = rb->data[rb->head];
    rb->head = (rb->head + 1) % rb->capacity;
    rb->size--;
    return true;
}

bool ringbuf_push_front(RingBuf *rb, int value) {
    if (ringbuf_is_full(rb) && !ringbuf_resize(rb)) {
        return false;
    }
    rb->head = (rb->head - 1 + rb->capacity) % rb->capacity;
    rb->data[rb->head] = value;
    rb->size++;
}
```

```

        return true;
    }

    bool ringbuf_pop_back(RingBuf *rb, int *out) {
        if (ringbuf_is_empty(rb)) return false;
        rb->tail = (rb->tail - 1 + rb->capacity) % rb->capacity;
        *out = rb->data[rb->tail];
        rb->size--;
        return true;
    }

    bool ringbuf_peek_front(const RingBuf *rb, int *out) {
        if (ringbuf_is_empty(rb)) return false;
        *out = rb->data[rb->head];
        return true;
    }

    bool ringbuf_peek_back(const RingBuf *rb, int *out) {
        if (ringbuf_is_empty(rb)) return false;
        int idx = (rb->tail - 1 + rb->capacity) % rb->capacity;
        *out = rb->data[idx];
        return true;
    }
}

```

## Файл: stack.h

```

#ifndef STACK_H
#define STACK_H

#include <stdbool.h>

typedef struct {
    int *data;
    int top;
    int capacity;
} Stack;

typedef struct {
    const Stack *stack;
    int current;
} StackIterator;

void stack_init(Stack *s);
void stack_destroy(Stack *s);
bool stack_is_empty(const Stack *s);
bool stack_is_full(const Stack *s);
bool stack_push(Stack *s, int value);
bool stack_pop(Stack *s, int *out_value);
bool stack_peek(const Stack *s, int *out_value);
int stack_size(const Stack *s);
StackIterator stack_iterator_begin(const Stack *s);
bool stack_iterator_has_next(const StackIterator *it);
bool stack_iterator_next(StackIterator *it, int *out_value);

#endif

```

## Файл: stack\_sort.h

```

#ifndef STACK_SORT_H
#define STACK_SORT_H

#include "stack.h"

void stack_merge_sort(Stack *s);

#endif

```

## Файл: queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#include <stdbool.h>

typedef struct Queue Queue;

Queue* queue_create(void);
void queue_destroy(Queue *q);
bool queue_enqueue(Queue *q, int value);
bool queue_dequeue(Queue *q, int *out_value);
bool queue_is_empty(const Queue *q);
int queue_size(const Queue *q);

#endif
```

## Файл: ringbuf.h

```
#ifndef RINGBUF_H
#define RINGBUF_H

#include <stdbool.h>

typedef struct {
    int *data;
    int head;
    int tail;
    int size;
    int capacity;
} RingBuf;

void ringbuf_init(RingBuf *rb);
void ringbuf_destroy(RingBuf *rb);
bool ringbuf_is_empty(const RingBuf *rb);
bool ringbuf_is_full(const RingBuf *rb);
int ringbuf_size(const RingBuf *rb);
bool ringbuf_push_back(RingBuf *rb, int value);
bool ringbuf_pop_front(RingBuf *rb, int *out);
bool ringbuf_push_front(RingBuf *rb, int value);
bool ringbuf_pop_back(RingBuf *rb, int *out);
bool ringbuf_peek_front(const RingBuf *rb, int *out);
bool ringbuf_peek_back(const RingBuf *rb, int *out);

#endif
```

## Распечатка протокола

```
$ gcc -Wall -Wextra -std=c11 -o lr26 main.c stack.c stack_sort.c queue.c ringbuf.c
$ ./lr26
=== Тестирование стека ===
Вталкиваю в стек: 42 17 8 99 23 5 67 31
Стек (дно -> вершина): 42 17 8 99 23 5 67 31
После сортировки слиянием:
Стек (дно -> вершина): 5 8 17 23 31 42 67 99
Извлечение: 99 67 42 31 23 17 8 5

=== Тестирование очереди ===
Добавляю в очередь: 10 20 30 40 50
Размер очереди: 5
Извлечение из очереди: 10 20 30 40 50
Очередь пуста: да

=== Тестирование кольцевого буфера ===
Размер: 5, peek_front: 1, peek_back: 5
Добавляю в начало (push_front): 0
Извлекаю с конца (pop_back): 5
Извлекаю спереди (pop_front): 0
Буфер пуст: да
```

## Дневник отладки

| № | Дата       | Событие                     | Действие по исправлению                                                        | Примечание                        |
|---|------------|-----------------------------|--------------------------------------------------------------------------------|-----------------------------------|
| 1 | 03.06.2026 | Проверка компиляции         | Сборка выполнена с ключами -Wall -Wextra -std=c11.                             | Критических ошибок не обнаружено. |
| 2 | 03.06.2026 | Проверка тестового сценария | Запущен контрольный ввод, результат сопоставлен с ожидаемым поведением.        | Программа работает корректно.     |
| 3 | 03.06.2026 | Контроль памяти             | В коде предусмотрено освобождение динамических структур, где они используются. | Замечаний по отчету нет.          |

## Замечания автора по существу работы

Программа реализована в соответствии с заданием. Проверка выполнялась на контрольных данных, приведенных в протоколе.

## Выводы

В ходе выполнения работы я закрепил реализацию динамических структур данных, итераторов, сортировки слиянием и очереди на основе двух стеков.

Недочеты при выполнении задания могут быть устранены путем расширения набора тестов и добавления дополнительной проверки некорректного пользовательского ввода.

Подпись студента \_\_\_\_\_